Data Mining

Model Overfitting

Introduction to Data Mining, 2nd Edition

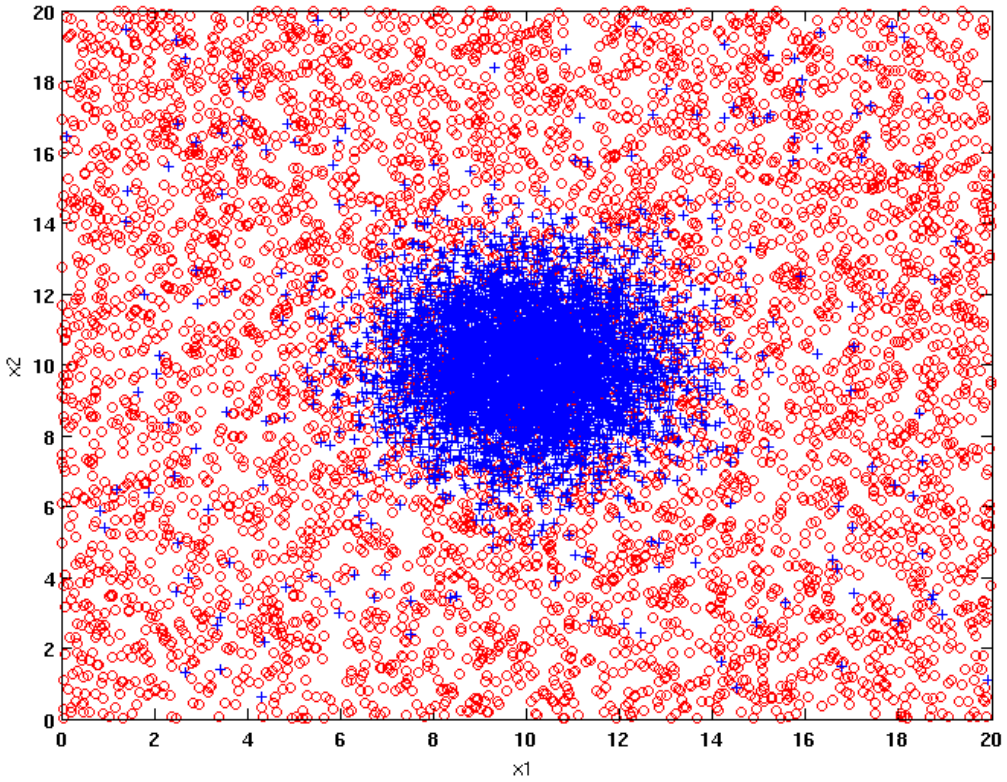
by

Tan, Steinbach, Karpatne, Kumar

Classification Errors

- Training errors (apparent errors)
  - Errors committed on the training set
- Test errors
  - Errors committed on the test set
- Generalization errors
  - Expected error of a model over random selection of records from same distribution

Example Data Set



Two class problem:

+ : 5400 instances

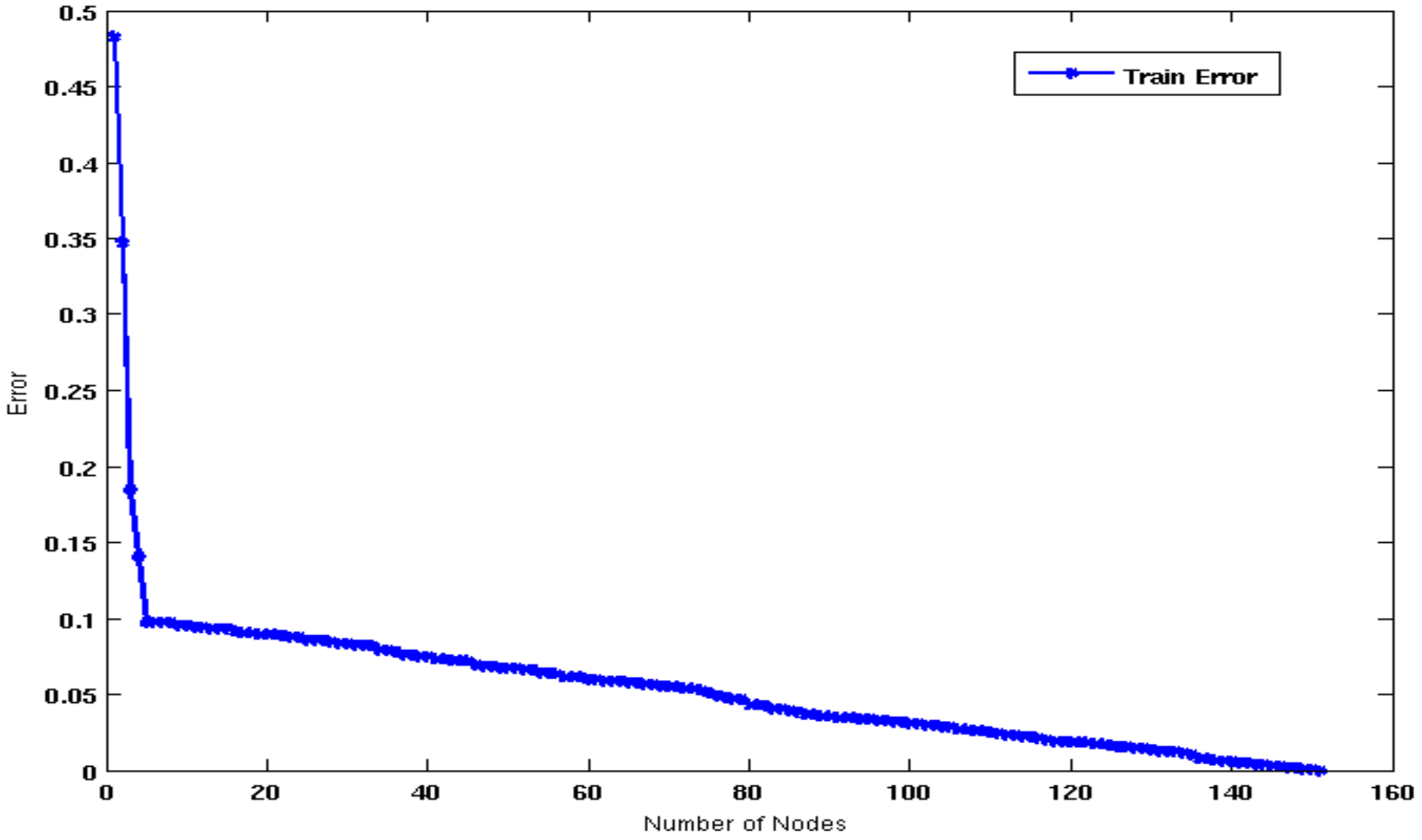
- **5000 instances generated from a Gaussian centered at (10,10)**
- **400 noisy instances added**

o : 5400 instances

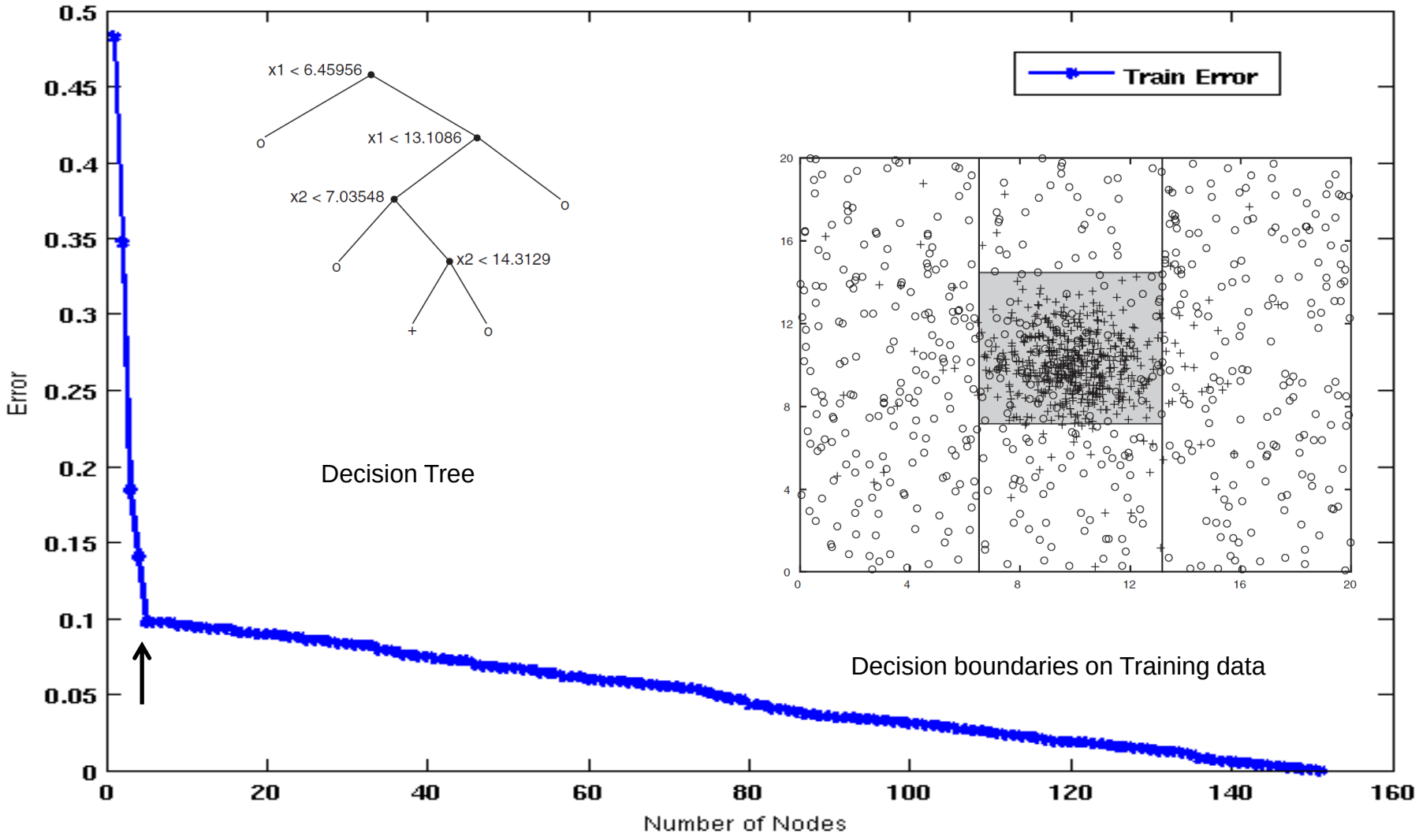
- **Generated from a uniform distribution**

10 % of the data used for training and 90% of the data used for testing

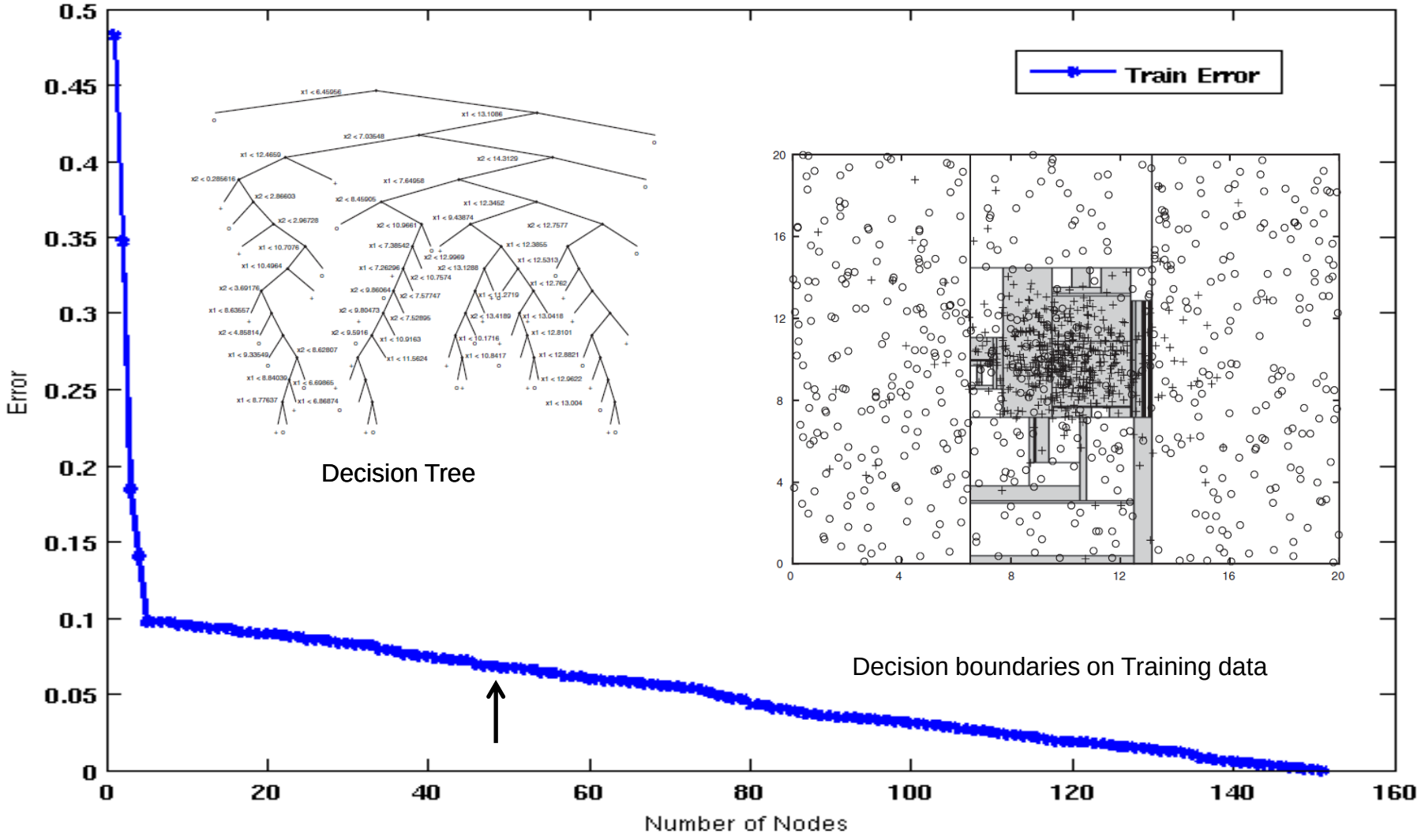
Increasing number of nodes in Decision Trees



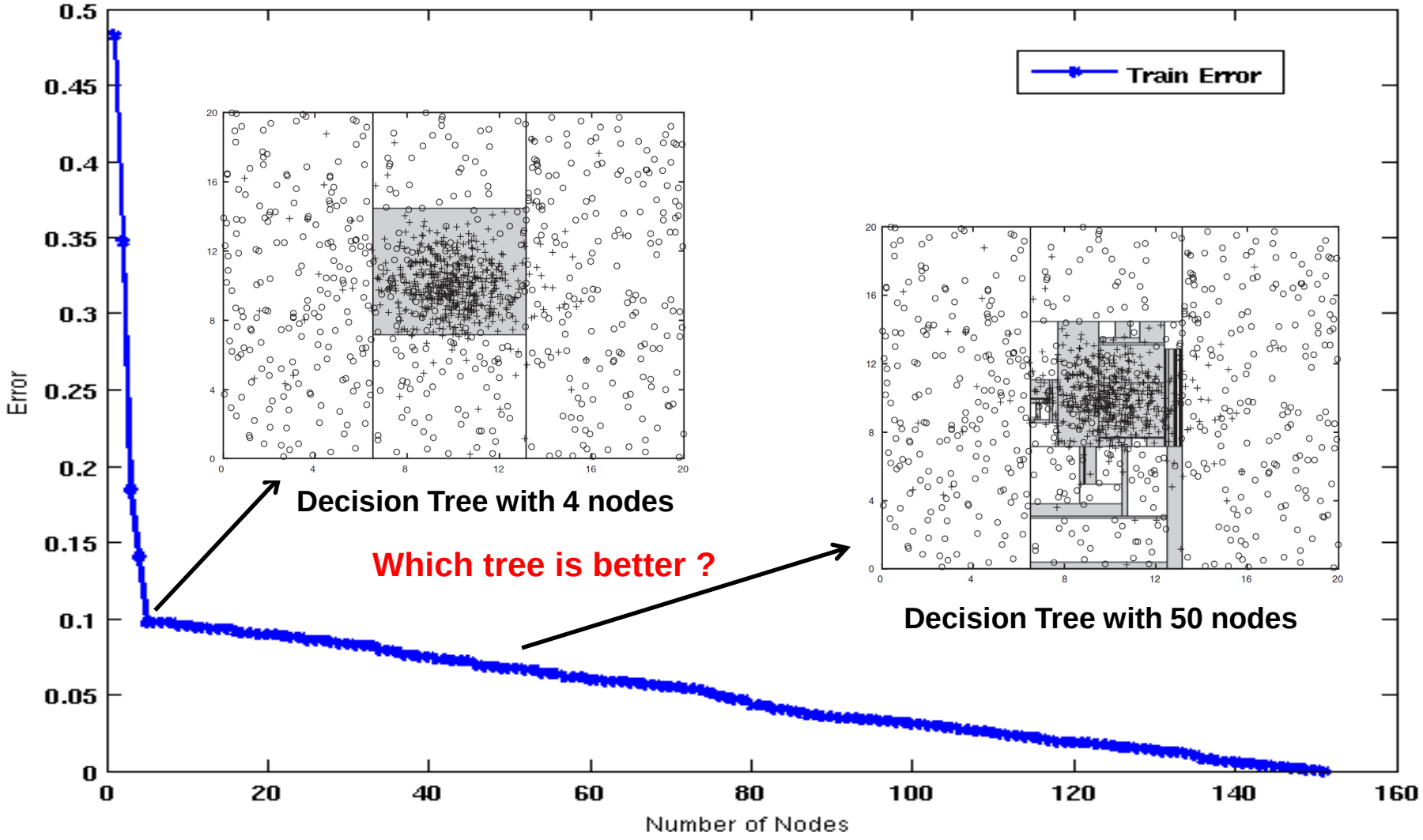
Decision Tree with 4 nodes



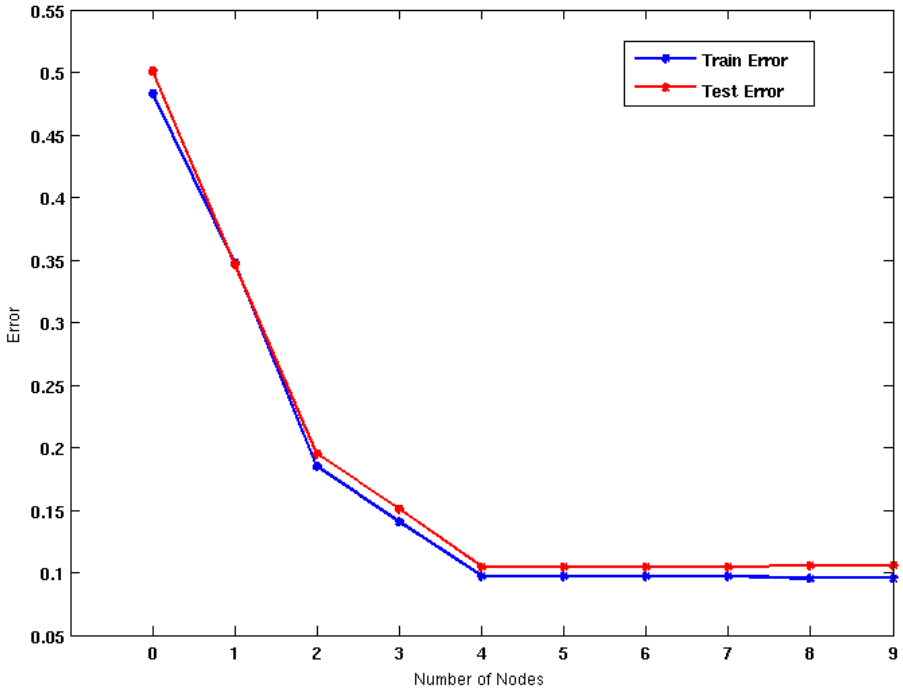
Decision Tree with 50 nodes

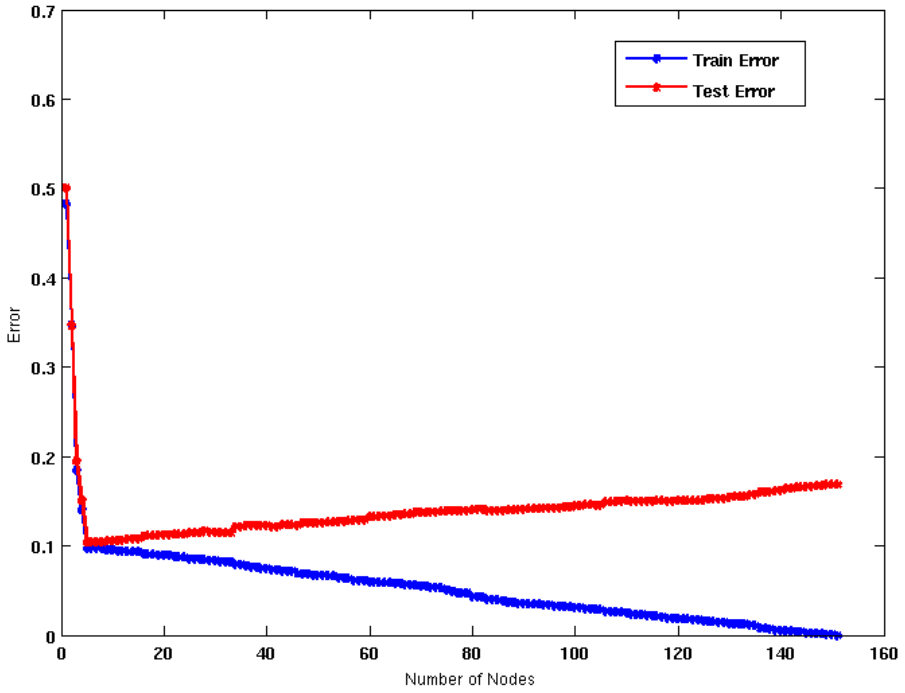


Which tree is better?



Model Overfitting



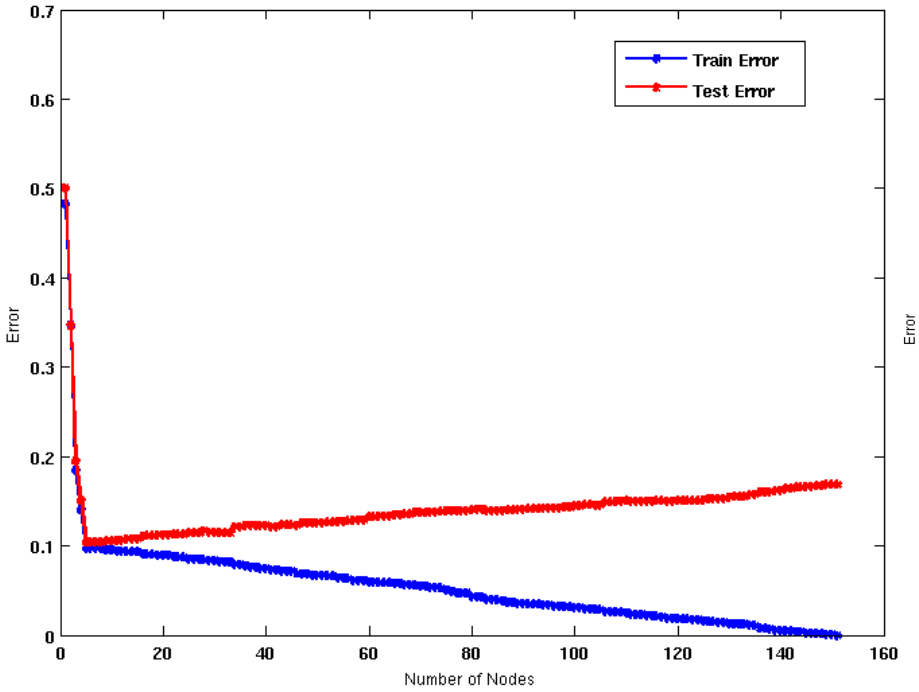


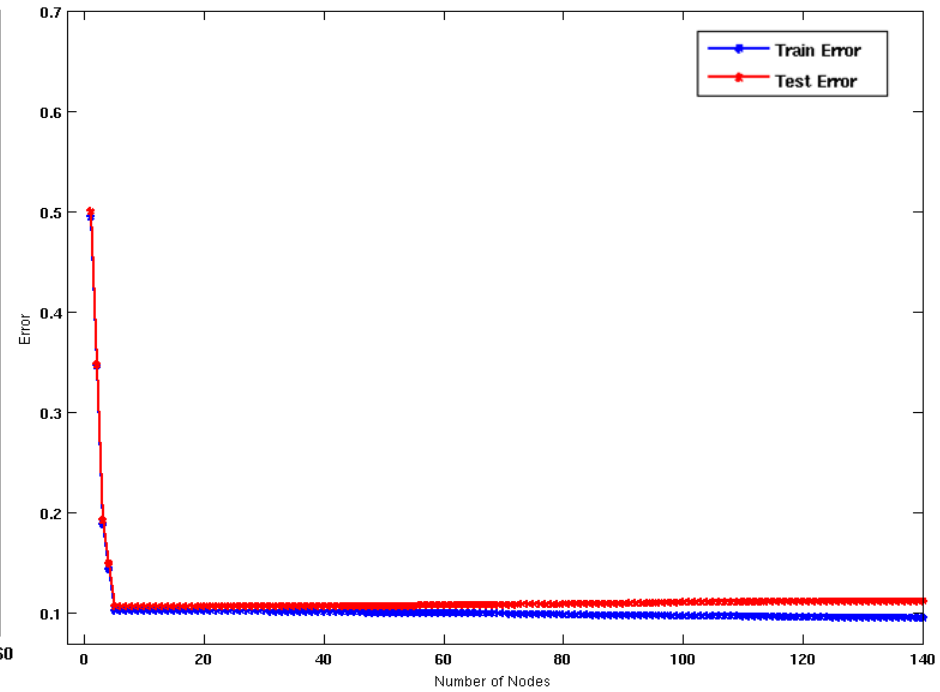
•As the model becomes more and more complex, test errors can start increasing even though training error may be decreasing

Underfitting: when model is too simple, both training and test errors are large

Overfitting: when model is too complex, training error is small but test error is large

Model Overfitting

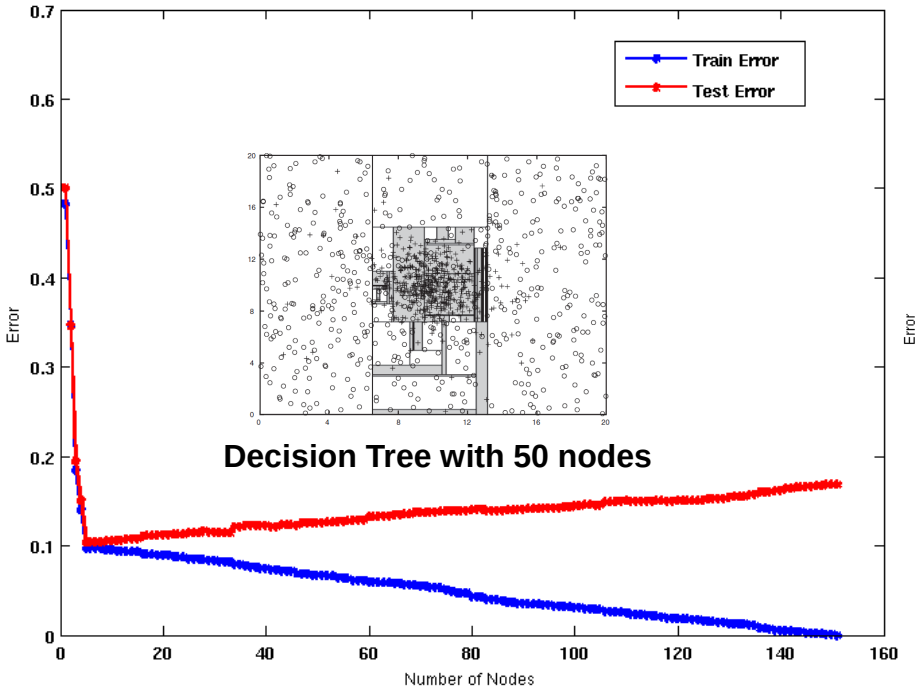


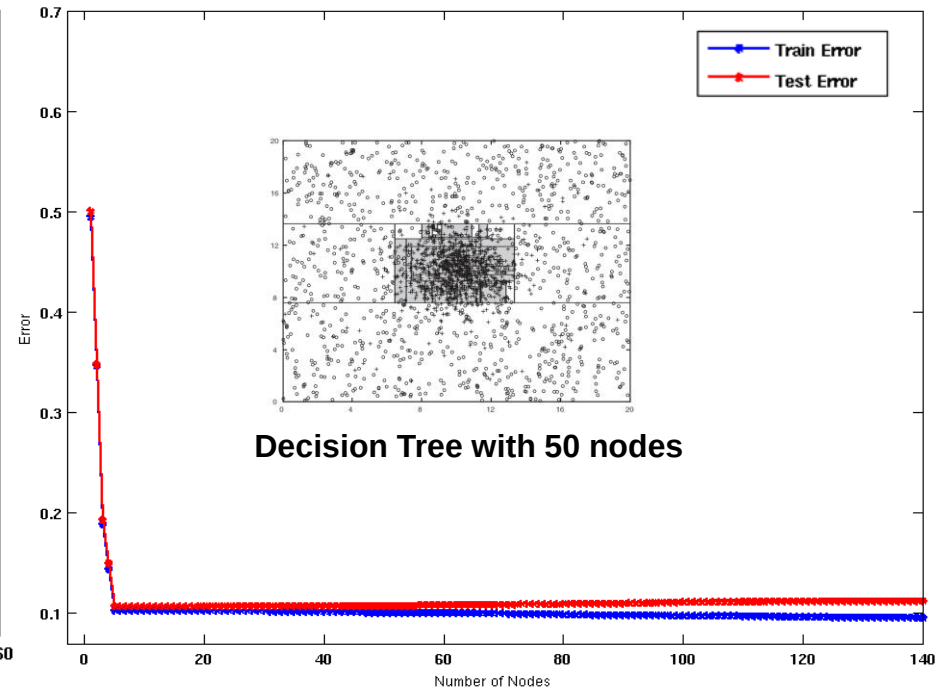


Using twice the number of data instances

- Increasing the size of training data reduces the difference between training and testing errors at a given size of model

Model Overfitting





Using twice the number of data instances

- Increasing the size of training data reduces the difference between training and testing errors at a given size of model

Reasons for Model Overfitting

- Limited Training Size

- High Model Complexity
  - Multiple Comparison Procedure

Effect of Multiple Comparison Procedure

- Consider the task of predicting whether stock market will rise/fall in the next 10 trading days

- Random guessing:

$$P(correct) = 0.5$$

- Make 10 random guesses in a row:

$$P(\#\,correct \geq 8) = \frac{\binom{10}{8} + \binom{10}{9} + \binom{10}{10}}{2^{10}} = 0.0547$$

Day 1	Up
Day 2	Down
Day 3	Down
Day 4	Up
Day 5	Down
Day 6	Down
Day 7	Up
Day 8	Up
Day 9	Up
Day 10	Down

Effect of Multiple Comparison Procedure

- Approach:
  - Get 50 analysts
  - Each analyst makes 10 random guesses
  - Choose the analyst that makes the most number of correct predictions

- Probability that at least one analyst makes at least 8 correct predictions

$$P(\#\,correct \geq 8) = 1 - (1 - 0.0547)^{50} = 0.9399$$

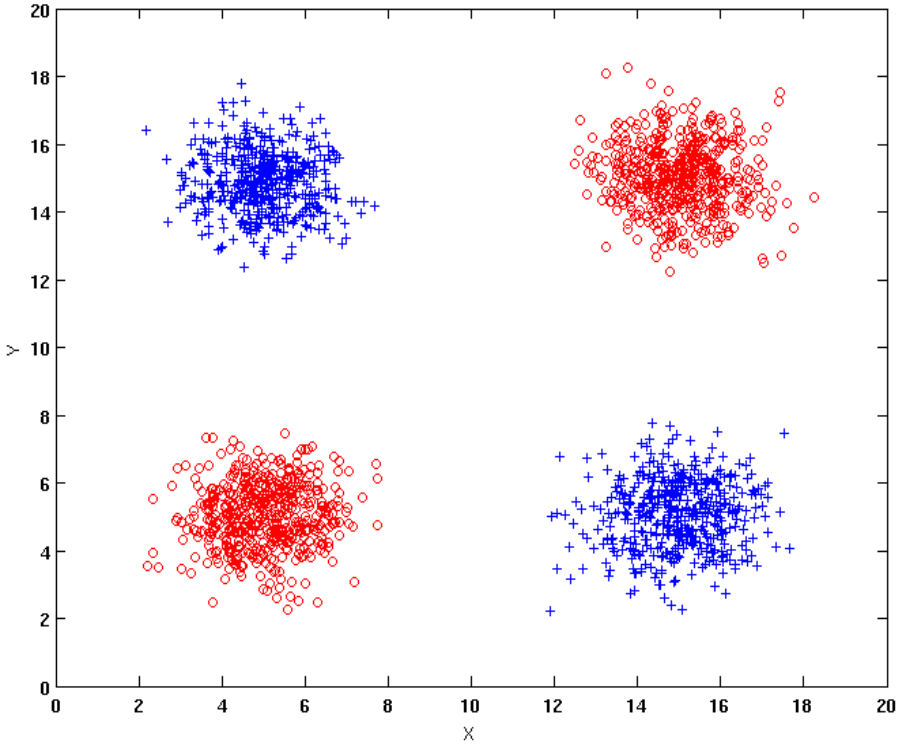
Effect of Multiple Comparison Procedure

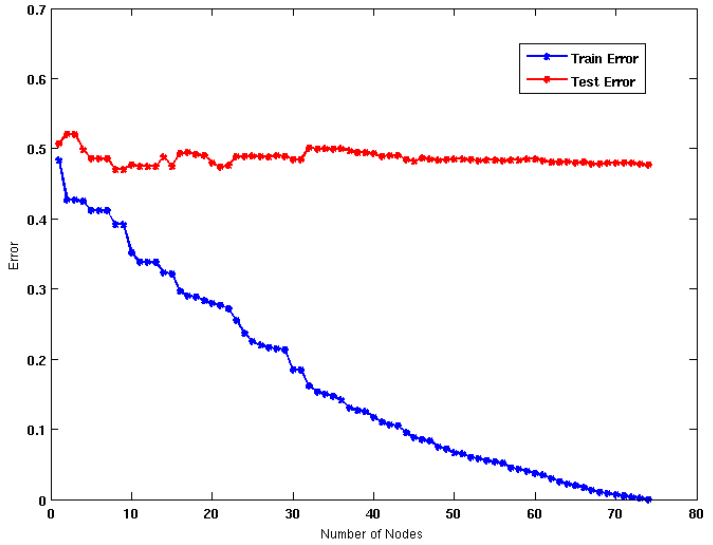
- Many algorithms employ the following greedy strategy:
  - Initial model: M
  - Alternative model: M' = M $\cup$ γ, where γ is a component to be added to the model (e.g., a test condition of a decision tree)
  - Keep M' if improvement, Δ(M,M') > α

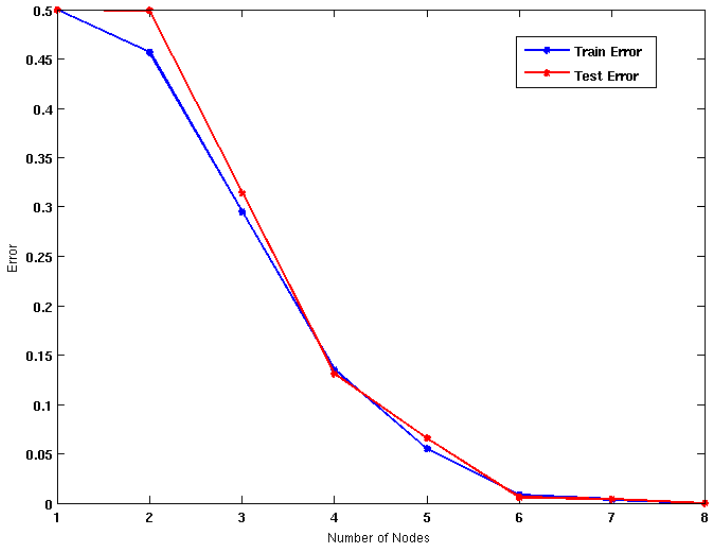
- Often times, γ is chosen from a set of alternative components, $\Gamma = \{\gamma_1, \gamma_2, \ldots, \gamma_k\}$

- If many alternatives are available, one may inadvertently add irrelevant components to the model, resulting in model overfitting

Effect of Multiple Comparison - Example







Use additional 100 noisy variables generated from a uniform distribution along with X and Y as attributes.

Use 30% of the data for training and 70% of the data for testing

Using only X and Y as attributes

Notes on Overfitting

- Overfitting results in decision trees that are more complex than necessary
- Training error does not provide a good estimate of how well the tree will perform on previously unseen records
- Need ways for estimating generalization errors

Model Selection

- Performed during model building
- Purpose is to ensure that model is not overly complex (to avoid overfitting)
- Need to estimate generalization error
  - Using Validation Set
  - Incorporating Model Complexity
  - Estimating Statistical Bounds

Using Validation Set

- Divide training data into two parts:
  - Training set:
    - use for model building
  - Validation set:
    - use for estimating generalization error
    - Note: validation set is not the same as test set

- Drawback:
  - Less data available for training

Model Selection:

Incorporating Model Complexity

- Rationale: Occam's Razor
  - Given two models of similar generalization errors, one should prefer the simpler model over the more complex model
  - A complex model has a greater chance of being fitted accidentally
  - Therefore, one should include model complexity when evaluating a model

Gen. Error(Model) = Train. Error(Model, Train. Data) + α x Complexity(Model)

weighted term of the complexity – the more complex the model, the greater the error

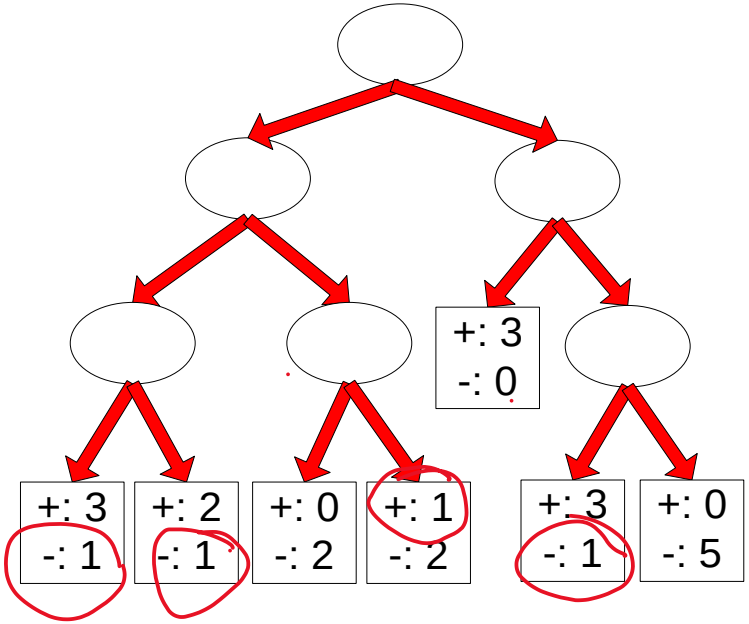
Estimating the Complexity of Decision Trees

- **Pessimistic Error Estimate** of decision tree T with k leaf nodes:

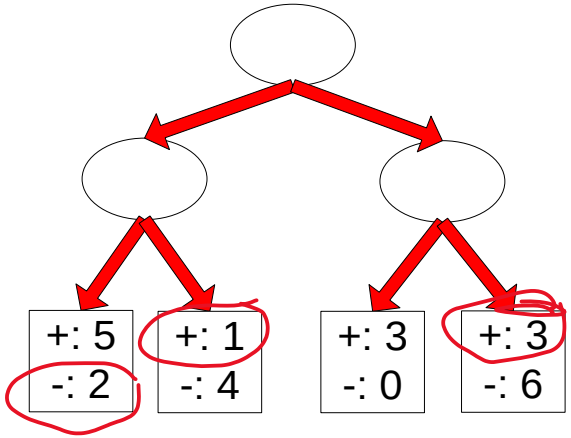
$$e_g(T) = \frac{\sum_{i=1}^{k} [e(t_i)+\Omega(t_i)]}{\sum_{i=1}^{k} n(t_i)} = \frac{e(T)+\Omega(T)}{N(T)}$$

- k: number of leaf nodes
- $e(t_i)$ - number of misclassified records in the leaf node; e(T): error rate on all training records
- Ω: trade-off hyper-parameter (similar to α), relative cost of adding a leaf node (each new node increases the complexity of the model)
  - For binary trees a penalty term of 0.5 means a node should always be expanded into its two child nodes as long as it improves the classification of at least one training record, because expanding a node, which is equivalent to adding 0.5 to the overall error, is less costly than committing one training error
- n(t_i) - number of training records classified by node t
- N(T) - total number of training records

Estimating the Complexity of Decision Trees: Example



Decision Tree, T_L



Decision Tree, T_R

$e(T_L) = 4/24$

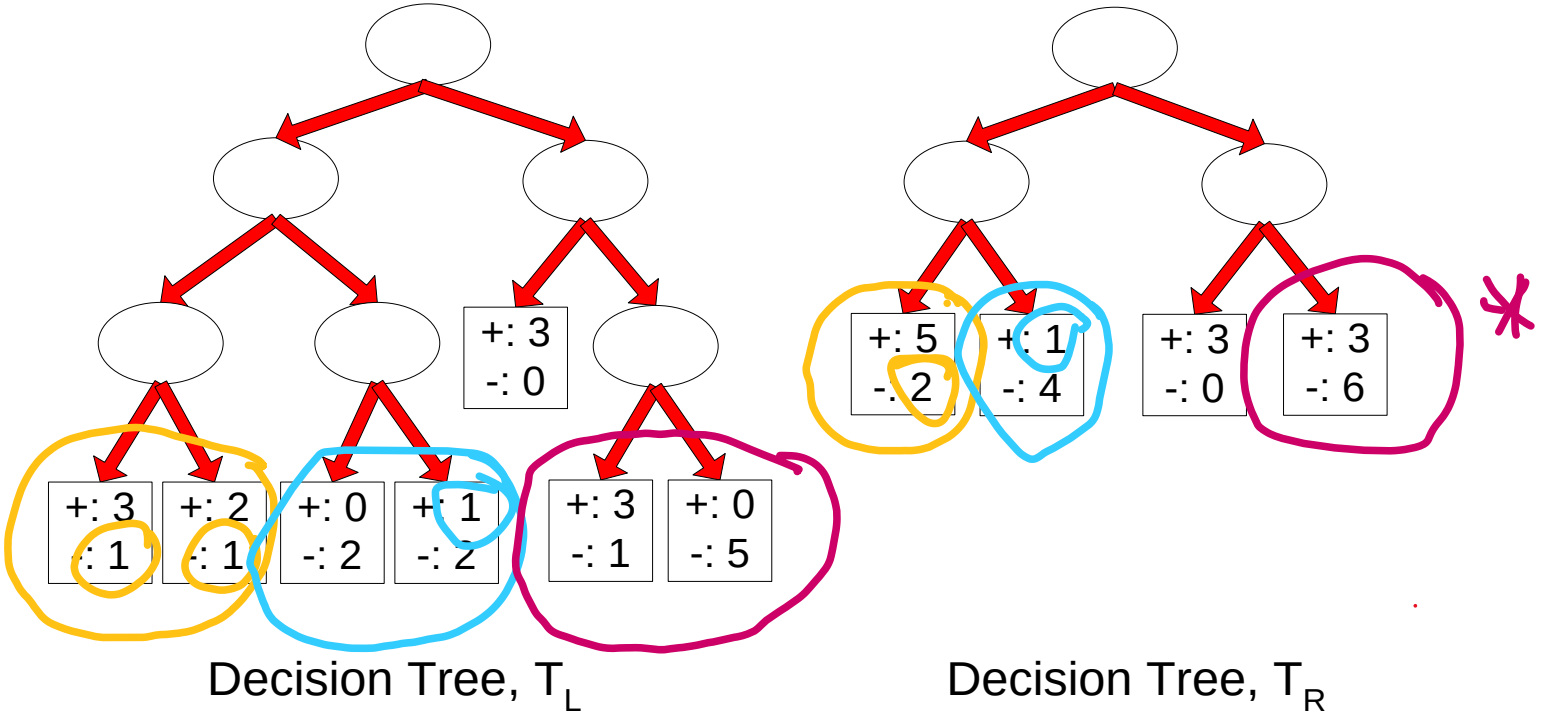
$e(T_R) = 6/24$

$\Omega = 1$

The number of misclassified records in the trees are 4 and 6 respectively (the records are classified by majority class)

$e_{gen}(T_L) = 4/24 + 1*7/24 = 11/24 = 0.458$
$e_{gen}(T_R) = 6/24 + 1*4/24 = 10/24 = 0.417$

Estimating the Complexity of Decision Trees: Example



Decision Tree, T_L Decision Tree, T_R

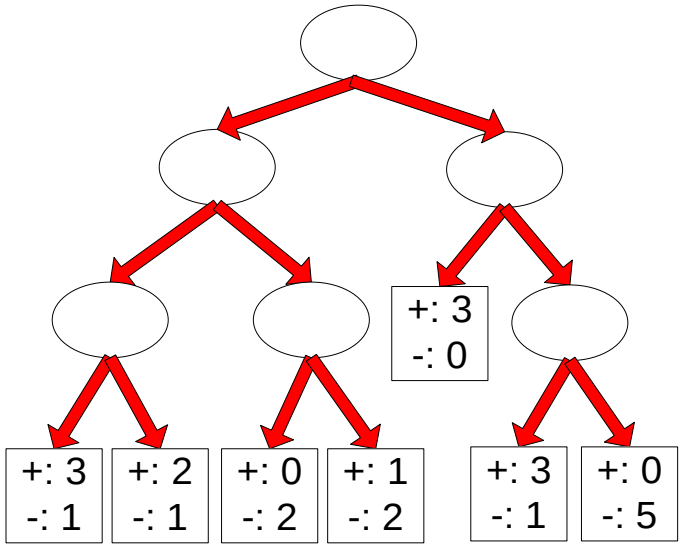
$e_{gen}(T_L) = 0.458$ vs $e_{gen}(T_R) = 0.417$

The right tree has a better pessimistic error than the left tree. Thus, a node should not be expanded into its child nodes unless it reduces the misclassification error for more than one training record.

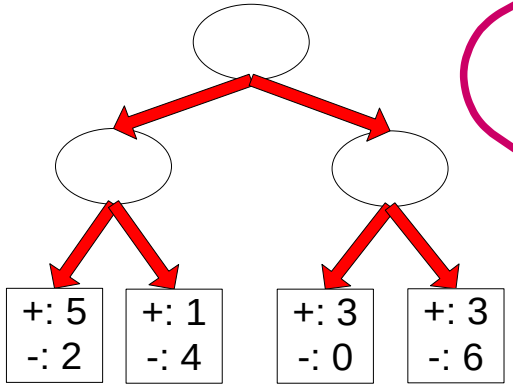
Calculate the pessimistic error of the tree obtained by expanding the right tree only in the node (e=0.375)

Estimating the Complexity of Decision Trees

- Resubstitution estimate approach assumes that training set is a good representation of the overall data. Thus the training error (also known as resubstitution error) is used as an optimistic estimate of generalization error. With this approach we will choose the model (tree) with the lowest training error. However, in general the training error is a poor estimate of the generalization error.



Decision Tree, T_L



Decision Tree, T_R

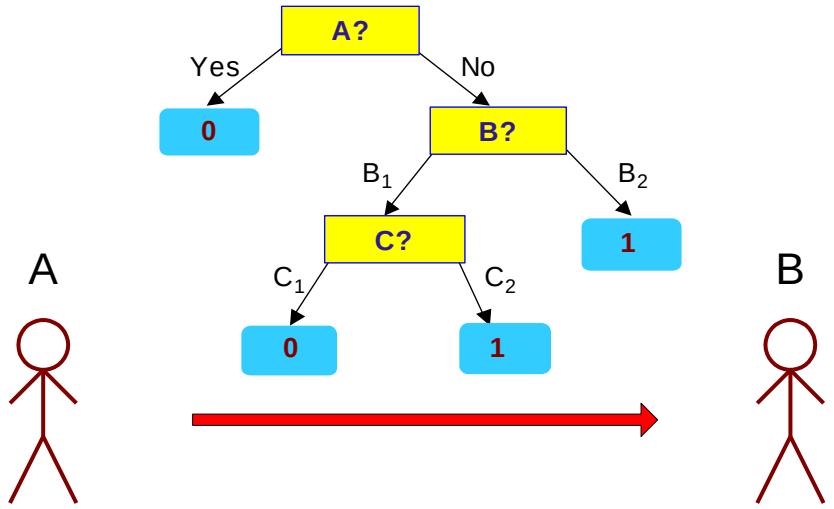
$e(T_L) = 4/24$

$e(T_R) = 6/24$

With this approach we will choose the left tree.

Minimum Description Length (MDL) (informative only)

X	y
X_1	1
X_2	0
X_3	0
X_4	1
…	…
X_n	1



X	y
X_1	?
X_2	?
X_3	?
X_4	?
…	…
X_n	?

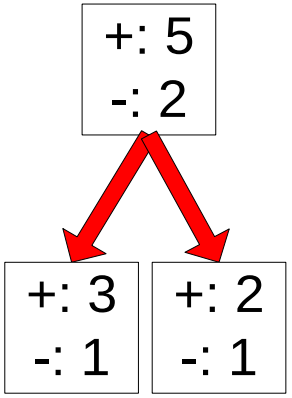
Concept from information theory (informative only, read from the book)

- Cost(Model,Data) = Cost(Data|Model) + α x Cost(Model)
  - Cost is the number of bits needed for encoding.
  - Search for the least costly model.
- Cost(Data|Model) encodes the misclassification errors.
- Cost(Model) uses node encoding (number of children) plus splitting condition encoding.

Estimating Statistical Bounds

The generalization error can be estimated as a statistical correction to the training error – usually this is an upper bound to the training error, taking into account the number of training records that reach a particular leaf node. An assumption in C4.5 is that number of errors in a leaf follows a binomial distribution, which can be approximated by normal distribution. The following formula is used

$$e'(N,e,\alpha) = \frac{e + \frac{z_{\alpha/2}^2}{2N} + z_{\alpha/2}\sqrt{\frac{e(1-e)}{N} + \frac{z_{\alpha/2}^2}{4N^2}}}{1 + \frac{z_{\alpha/2}^2}{N}}$$

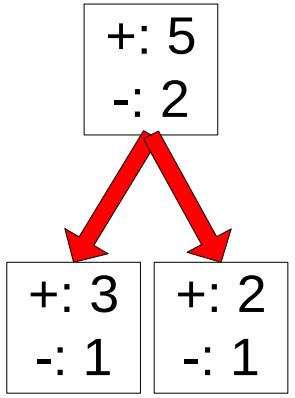


See the example of a subtree from the previous example.

Before splitting: the error rate of the node before splitting is
e = 2/7. For $\alpha = 25\%$ the upper bound of the error based on the formula is
e'(7, 2/7, 0.25) = 0.503
This corresponds to e'(T) = 7 $\times$ 0.503 = 3.521 errors

Estimating Statistical Bounds

$$e'(N,e,\alpha) = \frac{e + \frac{z_{\alpha/2}^2}{2N} + z_{\alpha/2}\sqrt{\frac{e(1-e)}{N} + \frac{z_{\alpha/2}^2}{4N^2}}}{1 + \frac{z_{\alpha/2}^2}{N}}$$



After splitting the node into its child nodes, the training errors for the child nodes are: ¼=0.25 and 1/3=0.333. The upper bounds for this errors based on the formula are

$e(T_L) = 1/4$, $e'(4, 1/4, 0.25) = 0.537$

$e(T_R) = 1/3$, $e'(3, 1/3, 0.25) = 0.650$

So the overall error of the child nodes is $e'(T) = 4 \times 0.537 + 3 \times 0.650 = 4.098$, which is larger than the estimated error before the splitting. Thus we should not split.

Model Selection for Decision Trees

- Pre-Pruning (Early Stopping Rule)
  - Stop the algorithm before it becomes a fully-grown tree
  - Typical stopping conditions for a node:
    - Stop if all instances belong to the same class
    - Stop if all the attribute values are the same
  - More restrictive conditions:
    - Stop if number of instances is less than some user-specified threshold
    - Stop if class distribution of instances are independent of the available features (e.g., using χ^2 test)
    - Stop if expanding the current node does not improve impurity measures (e.g., Gini or information gain).
    - Stop if estimated generalization error falls below certain threshold

Model Selection for Decision Trees

- Post-pruning
  - Grow decision tree to its entirety
  - Subtree replacement
    - Trim the nodes of the decision tree in a bottom-up fashion
    - If generalization error improves after trimming, replace sub-tree by a leaf node
    - Class label of leaf node is determined from majority class of instances in the sub-tree
  - Subtree raising
    - Replace subtree with most frequently used branch

Example of Post-Pruning

Class = Yes	20
Class = No	10
Error = 10/30	

Training Error (Before splitting) = 10/30

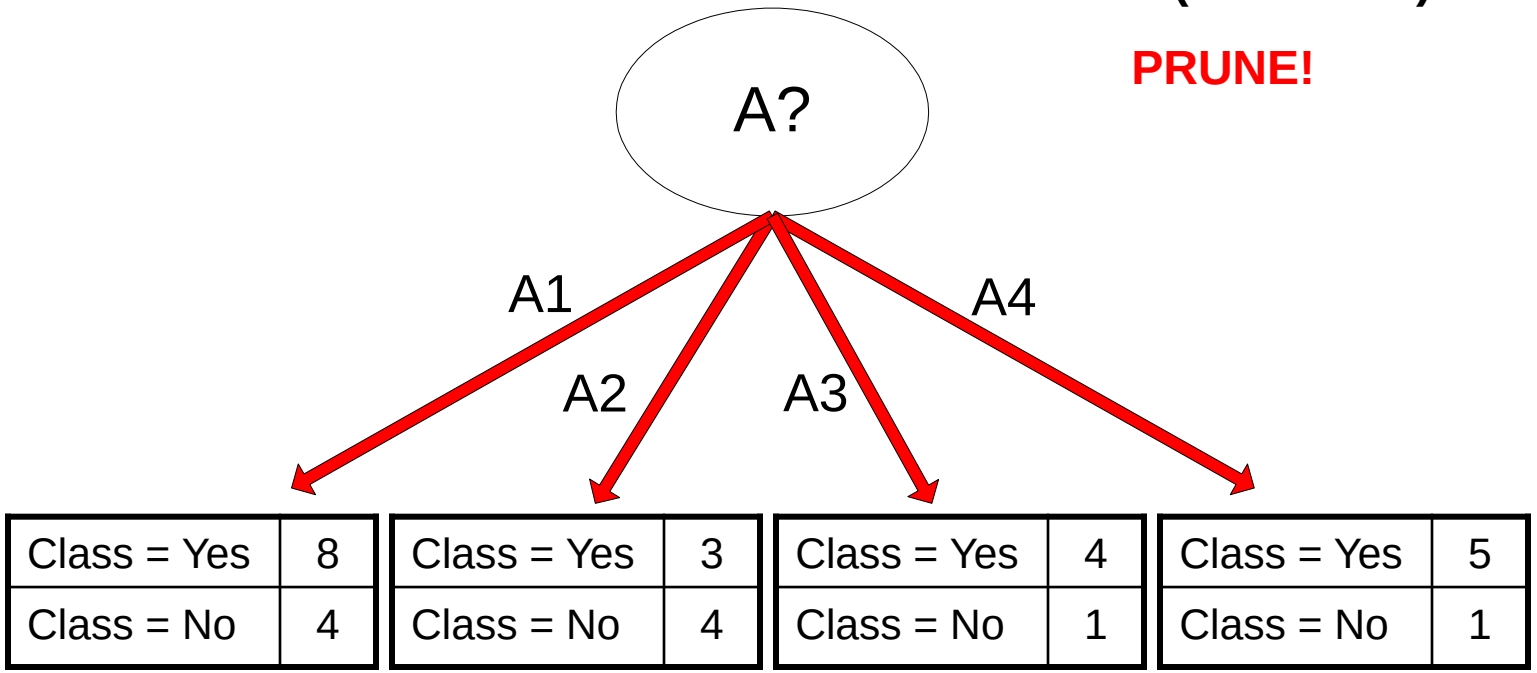
Pessimistic error = (10 + 0.5)/30 = 10.5/30

Training Error (After splitting) = 9/30

Pessimistic error (After splitting)

= (9 + 4 × 0.5)/30 = 11/30

PRUNE!



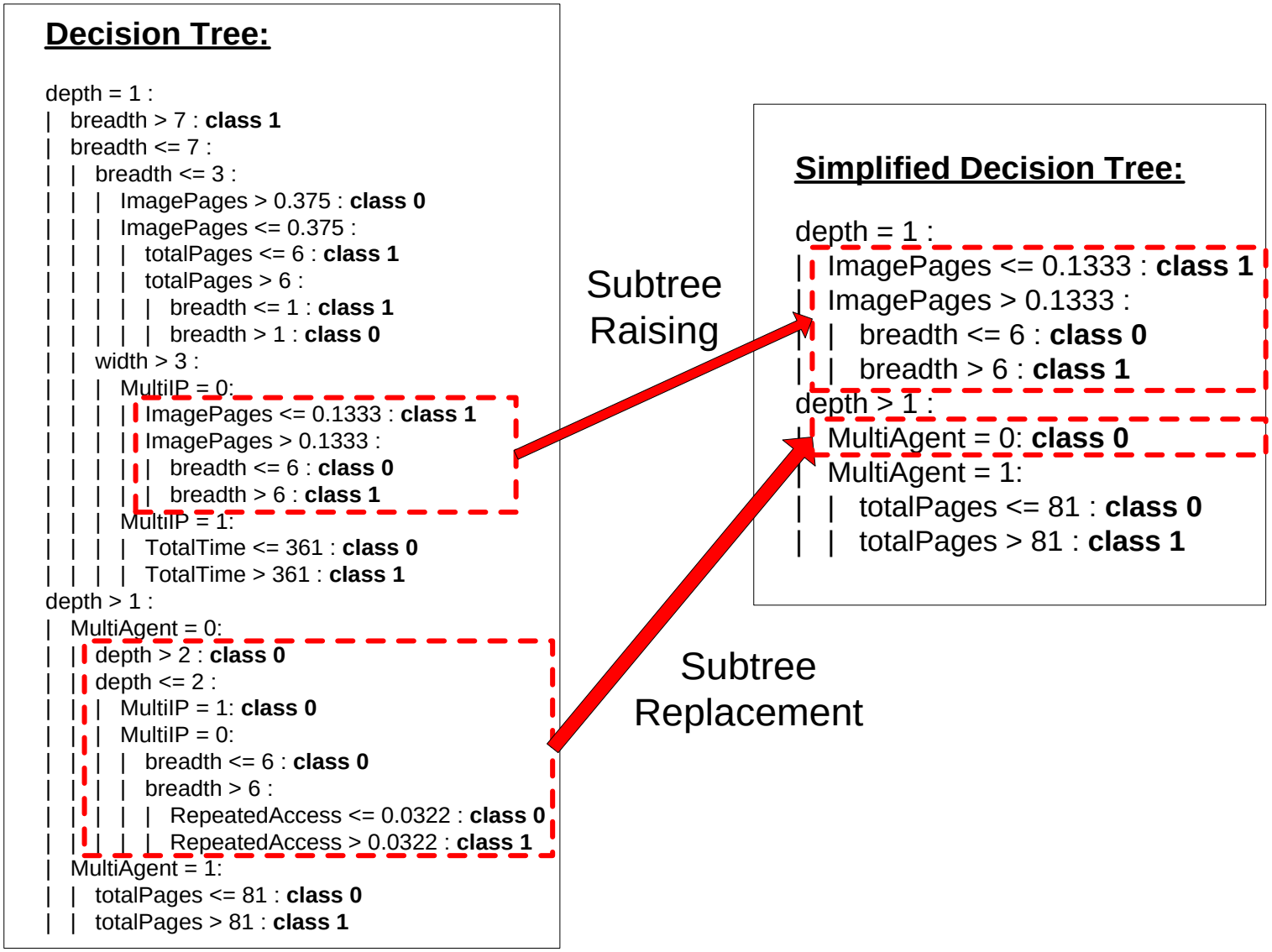
Class = Yes	8
Class = No	4

Class = Yes	3
Class = No	4

Class = Yes	4
Class = No	1

Class = Yes	5
Class = No	1

Examples of Post-pruning

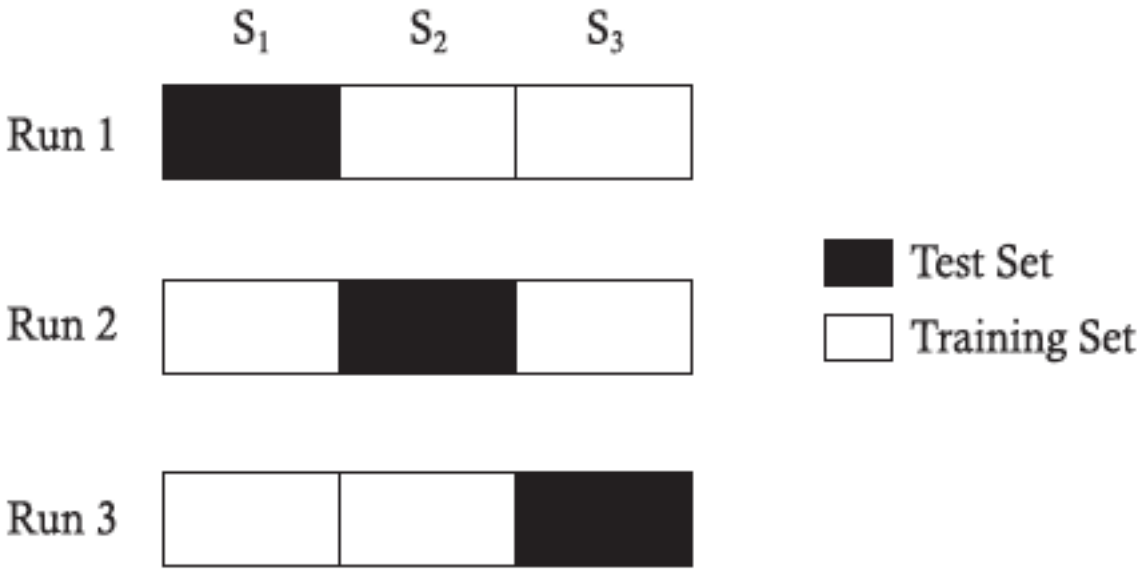


Model Evaluation

- Purpose:
  - To estimate performance of classifier on previously unseen data (test set)
- Holdout
  - Reserve k% for training and (100-k)% for testing
  - Random subsampling: repeated holdout
- Cross validation
  - Partition data into k disjoint subsets
  - k-fold: train on k-1 partitions, test on the remaining one
  - Leave-one-out: k=n

Cross-validation Example

- 3-fold cross-validation



Variations on Cross-validation

- Repeated cross-validation
  - Perform cross-validation a number of times
  - Gives an estimate of the variance of the generalization error
- Stratified cross-validation
  - Guarantee the same percentage of class labels in training and test
  - Important when classes are imbalanced and the sample is small
- Use nested cross-validation approach for model selection and evaluation